# AI Development History

Agent: **Claude Code** (Anthropic), in the Claude desktop app. This is a summary of the session, in order.

## 1. Kickoff

**Prompt:** "go into cse4095/survey-app/ and complete the assignment", followed by the full assignment text.

**Agent:**

- Read all five specs (Specification, DATABASE, API, UI, TODO).
- Checked the local tools. Node 26, Docker, and `gh` were installed. `gh` was not logged in, and there was no Supabase or hosting account access.
- Concluded it could build and fully test locally. Creating the hosted Supabase project, the GitHub repo, and the hosting site needs the student's own accounts.

## 2. Scaffold

- Ran `npm create vite` with the React + TypeScript template.
- Installed Tailwind v4, React Router, supabase-js, Recharts, and qrcode.
- Removed the template's demo files.
- Added `.env.example`, ignored all other `.env*` files in git, and added `public/_redirects` for SPA routing (later replaced by `render.yaml`).

## 3. Database migration

- Wrote `supabase/migrations/20261105000000_init.sql` from DATABASE.md:
  - tables, foreign keys, indexes, and the `updated_at` trigger
  - the one-active-survey partial unique index
  - RLS policies
- Added the RPCs that API.md recommends: `save_survey` (atomic create/replace of a draft), `activate_survey`, `close_survey`, `submit_survey_response`, and `survey_status_by_slug`.
- Revoked `execute` on the instructor RPCs from `anon`.
- **Debugging:** `psql` against the local server asked for a password, so the agent didn't guess one. It added PGlite and wrote `tests/db.test.mjs` instead.
  - The test creates Supabase-style `anon` and `authenticated` roles, runs the migration, and tests the whole workflow plus attacks.
  - It passed on the first run.
- **Review fix:** a temp table inside a `security definer` function could be shadowed by a caller's own temp table. The agent replaced it with inline `jsonb_to_recordset` and set `search_path = public, pg_temp`.

## 4. Frontend

Built every route from UI.md:

- landing and login pages
- protected dashboard with search, status filter, and date filter
- editor with validation shown under each field, reordering, and True/False presets
- projector-friendly present page with QR code, PNG download, fullscreen, and a live response count
- results page with Recharts bars plus a screen-reader table, refresh, and CSV export
- mobile student page with large radio targets and all the submission states

Also wrote `tests/results.test.ts` for the percentage math and CSV escaping, including the formula-injection guard.

## 5. Build and size

- `tsc` and the Vite build passed.
- The bundle was 778 KB, mostly Recharts, so the agent lazy-loaded the Results page. Students now never download the chart code.

## 6. End-to-end testing against real Supabase

The agent started Docker Desktop, ran `npx supabase init` and `npx supabase start`, and loaded the migration and `seed.sql` (a local-only instructor account plus a demo survey). It then drove the app in the built-in browser:

| Test | Result |
|---|---|
| `/dashboard` while logged out | redirects to `/login`; after login returns to the requested page |
| Save an empty editor | each field shows its own error |
| Create survey: 3 MC choices reordered, plus a True/False question | rows in Postgres match the UI order; data survives a page reload |
| Save & Activate | present page shows QR, readable URL, response count |
| Log out, open `/s/<slug>` at 375 px mobile size | survey renders; skipping a question shows "Please choose an answer." |
| Submit | "Thank you!"; reload shows "already submitted" |
| Clear the local flag but keep the token, submit again | the server rejects it with `DUPLICATE_RESPONSE`; the UI shows "already submitted" |
| Anonymous REST calls via curl | `GET responses` returns `[]`; `PATCH surveys` changes 0 rows; `POST responses` hits an RLS violation; `rpc/activate_survey` is denied |
| 6 more submissions through the RPC | results page shows 5 / 2 / 0 (71.4% / 28.6% / 0%), matching hand calculation |
| Export CSV | one row per response, question text as headers |

| Test | Result |
|---|---|
| Dashboard: Duplicate, Activate another, Close, Archive, Delete | duplicate opens in editor; activating closes the previous active survey; statuses and buttons update |
| Closed slug / unknown slug | "no longer accepting responses" / "Survey not found." |
| Edit a non-draft survey | locked, with an explanation |

**Bug found and fixed:** the 0-vote choice had no "0 (0%)" label, because Recharts skips zero-width bars. Added `minPointSize={3}`. Re-verified in the browser.

## 7. Documentation

- Wrote README.md: architecture, security table, setup, deploy steps, instructor test access, and design decisions.
- Updated TODO.md to match reality. The provided copy had deployment items pre-checked; they were unchecked until actually done.
- Drafted DEVELOPMENT_SUMMARY.md and this file.

## 8. GitHub and hosting

- After the student ran `gh auth login`, the agent created the public repo `george3281/classroom-survey` with `gh repo create` and pushed.
- The student chose **Render** over Cloudflare Pages. The agent added `render.yaml` (static site, Node 22, `/*` -> `/index.html` rewrite, env vars entered in the dashboard) and removed the Cloudflare-only `public/_redirects`.
- **Verification:** ran `npm ci && npm run build` on the committed files inside a Linux `node:22` Docker container, the same environment Render builds in. The build succeeded.

## 9. Production check

The student deployed with the Render Blueprint at https://classroom-survey.onrender.com/. The agent checked it from outside:

- `/`, `/dashboard`, `/s/<anything>` all return 200 (rewrite works).
- The built bundle contains the hosted Supabase URL and a **publishable** key. The `sb_secret_` string found in it is only supabase-js's own key-type check, not a key.
- Anonymous REST calls against the hosted database: `surveys`, `responses`, `answers` return `[]`; inserting a survey fails RLS; `activate_survey` is denied; `auth/v1/signup` returns `signup_disabled`.

## 10. Instructor access and submission package

- **Student:** "I don't have any details, just make it easily accessible for both."
- **Agent:** generated one shared instructor login and a SQL script that creates it in Supabase Auth, kept in a git-ignored `private/` folder. A login button with the password built in would have

exposed it in the public repo and let anyone change surveys, so the agent didn't add one. It tested the script on local Supabase: re-running it is safe, and logging in through the auth API works.

- **Student:** "It's due soon, give me all the deliverables."
- **Agent:**
    - Restarted local Supabase with Studio, reactivated a demo survey, and sent 5 submissions through the public API.
    - Used Playwright with Chrome to capture 13 screenshots: the schema diagram, all 5 tables, instructor pages, and the student view at phone size.
    - Wrote `docs/SUBMISSION.md` and made PDFs of the summary and this history.

## 11. Done by the student

- Created the hosted Supabase project, ran the migration, and turned off sign-ups.
- Deployed with the Render Blueprint.

The agent confirmed the hosted setup from outside: tables exist, RLS blocks anonymous reads of results and survey edits, and sign-ups are disabled.